

Aula 11 — I²C e memória não volátil

A mesma tarefa, por dois mecanismos

Microcontroladores — DENE/UFMT

OBJETIVOS

- Guardar e recuperar um valor entre reinicializações, usando a memória interna por registradores e a externa por barramento.
- Explicar o dreno aberto e justificar a necessidade dos resistores de elevação.
- Descrever o byte de controle e argumentar por que o endereço viajar no barramento é o que dispensa fios dedicados.
- Reconhecer o start repetido como prova de que escrita e leitura são fases de uma mesma transação.
- Comparar espera cega com confirmação pelo dispositivo, e estimar o custo de cada uma.
- Dimensionar a frequência de escrita a partir da vida útil da memória.

1 O que não sobrevive ao reset

O usuário ajustou o alvo para 42 °C no botão do encontro 7. O aquecedor ligou, a tensão caiu, e o encontro 8 mostrou o que acontece: reset por subtensão. O programa recomeça, e o alvo volta a ser o que estava escrito no código.

Toda variável deste curso mora na RAM, e a RAM é volátil por construção — ela guarda carga, e carga sem alimentação se dissipa em milissegundos.

CONCEITO

Guardar algo entre reinicializações exige uma tecnologia diferente, e o PIC18F4550 dá acesso a duas: uma dentro do chip e outra fora dele. Elas resolvem o mesmo problema por caminhos completamente diferentes, e é essa comparação que organiza a aula.

2 Dentro do chip: registradores num endereço

A memória não volátil interna tem 256 bytes e nenhum barramento. Fala-se com ela como se fala com qualquer periférico do encontro 1: escrevendo em registradores de função especial.

```
uint8_t eeprom_ler(uint8_t end)
{
    EEADR = end;
    EECON1bits.EEPGD = 0;      /* memoria de dados, nao a Flash */
    EECON1bits.CFGS = 0;      /* nao os bits de configuracao */
    EECON1bits.RD = 1;
    return EEDATA;            /* pronto no ciclo seguinte */
}

void eeprom_escrever(uint8_t end, uint8_t dado)
{
    EEADR = end;
    EEDATA = dado;
    EECON1bits.EEPGD = 0;
    EECON1bits.CFGS = 0;
    EECON1bits.WREN = 1;      /* habilita escrita */

    uint8_t salvo = INTCONbits.GIE;
    INTCONbits.GIE = 0;      /* a sequencia abaixo nao pode ser interrompida */
    EECON2 = 0x55;
    EECON2 = 0xAA;
```

```

EECON1bits.WR = 1;
INTCONbits.GIE = salvo;

while (EECON1bits.WR) { } /* cerca de 4 ms */
EECON1bits.WREN = 0;      /* desabilita de novo */
}

```

A sequência 0x55, 0xAA existe por uma razão específica. EEC0N2 não é um registrador de verdade: não guarda nada e não pode ser lido. Ele é uma fechadura.

CONCEITO

Sem ela, um programa que perdeu o rumo — ponteiro errado, pilha corrompida, retorno para um endereço qualquer — poderia escrever na memória não volátil por acidente, e destruir a calibração ou a configuração do produto de campo.

Com ela, o acidente teria de executar exatamente três escritas na ordem certa, sem nada no meio. É proteção contra código descontrolado, e é por isso que `INTCONbits.GIE` precisa ser desligado ali: uma interrupção entre as duas metades da chave anula a sequência.

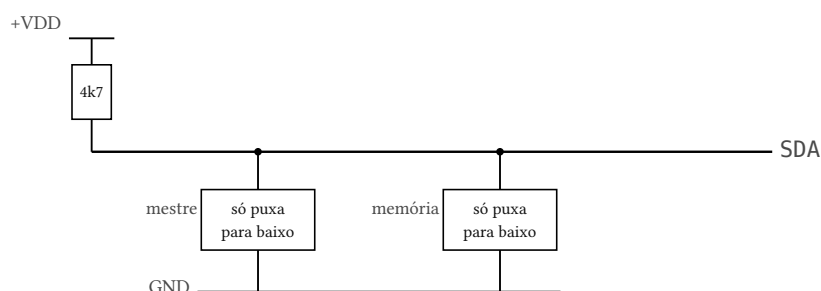
Repare no `while (EECON1bits.WR)`. São 4 ms de espera cega — três vezes uma atualização de tela. Escrever na memória não volátil é a operação mais lenta que este curso encontrou até agora, e ela vai ficar mais lenta ainda na versão externa.

NOTA

3 Fora do chip: um contrato de barramento

A mesma tarefa, agora por dois fios.

3.1 Dreno aberto



Ninguém empurra a linha para cima — quem faz isso é o resistor. Basta um dispositivo puxar para que a linha fique em zero, e é por isso que dois transmissores simultâneos não queimam nada.

Figura 1: Nenhum dispositivo empurra a linha para cima. Isso é o que permite que vários falem no mesmo par de fios sem risco de curto.

Cada dispositivo tem apenas um transistor que puxa a linha para o terra. Ninguém tem como forçá-la para cima; quem faz isso é um resistor.

CONCEITO

A consequência é uma função lógica cablada: a linha está em 1 se e somente se **todos** estiverem soltos. Se dois dispositivos falarem ao mesmo tempo, o que disser 0 vence, e nada queima — enquanto num barramento com saídas empurrando dos dois lados isso seria um curto-circuito entre a alimentação e o terra.

O preço é a velocidade. A subida da linha é feita por um resistor carregando uma capacitância, e não por um transistor: por isso o barramento é assimétrico, com descida rápida e subida lenta, e por isso existe um limite de capacitância total que restringe quantos dispositivos e quanto fio o barramento aceita.

A verificar antes do R11. A placa já traz os resistores de elevação em SDA e SCL, ou o roteiro precisa incluí-los? E qual é a peça de memória — 24C02, 24C04 ou 24C16?

NESTE KIT

A resposta muda a aula: o tamanho da página, a quantidade de bits de endereço de dispositivo e a existência de bits de seleção de bloco dependem da peça.

3.2 O endereço viaja no barramento

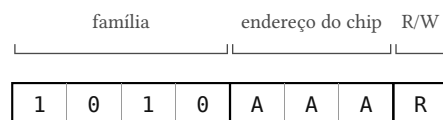


Figura 2: O byte de controle. Quatro bits identificam a família de memórias, três escolhem o chip, e o último diz o sentido da transação.

No barramento paralelo do encontro 3, a linha RS era um fio dedicado a dizer **o que** o dado significava. Aqui não há fio dedicado: o endereço é o primeiro byte que trafega.

CONCEITO

É essa a resposta para a pergunta que o encontro 3 deixou aberta — por que o paralelo precisa de tantos fios. Ele precisa porque coloca no espaço o que o serial coloca no tempo.

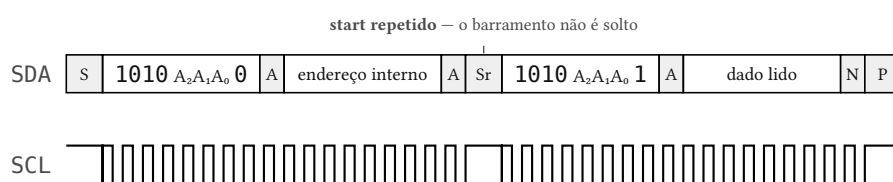
O erro número um de quem começa em I²C. O endereço tem sete bits; o que trafega é um byte de oito. A folha de dados de um sensor diz “endereço 0x48”; o que precisa ir para o barramento é 0x48 deslocado uma casa à esquerda, mais o bit de sentido — ou seja, 0x90 para escrever e 0x91 para ler.

ATENÇÃO

Quem escrever 0x48 direto está endereçando o dispositivo 0x24 em modo de leitura. Não há dispositivo nenhum ali, ninguém responde, e o sintoma é ausência de reconhecimento — que se parece com fio solto, com pull-up faltando e com memória queimada.

Metade das bibliotecas espera o endereço de sete bits e desloca por você; a outra metade espera o byte já montado. Ler a documentação da biblioteca aqui não é opcional.

3.3 A transação, e o start repetido



Uma única transação, com dois sentidos. O endereço interno é **escrito**, e só então o sentido se inverte — sem que ninguém mais possa entrar no meio.

Figura 3: Ler um byte de um endereço qualquer da memória. Note que existe **uma** transação, e não duas.

Para ler de um endereço arbitrário é preciso primeiro **escrever** esse endereço na memória, e só então inverter o sentido. A tentativa é fazer isso como duas operações: escreve o endereço, encerra, começa de novo e lê.

CONCEITO

Não funciona de forma confiável. Entre as duas, o barramento fica livre, e outro mestre pode assumir e mexer no ponteiro interno da memória.

O start repetido resolve: emite-se uma nova condição de início **sem** soltar o barramento. Escrita e leitura passam a ser duas fases de uma mesma transação indivisível.

Guarde a forma deste argumento. É a mesma da leitura de dezesseis bits do encontro 7: uma operação que parece atômica é composta por partes, e o defeito aparece quando alguém entra no meio.

NOTA

Lá o intruso era a interrupção; aqui é outro mestre no barramento. A solução tem o mesmo formato — tornar a sequência indivisível.

3.4 O custo, e a espera que acabou

Encontro	Barramento	Endereçamento	Confirmação
2	pino direto	—	—
3	paralelo (HD44780)	RS seleciona o registrador	nenhuma — espera cega
10	UART	— (ponto a ponto)	nenhuma
11	I ² C	7 bits, no próprio barramento	ACK a cada byte

Depois de receber um byte, a memória puxa a linha para baixo por um tempo de bit. Esse é o reconhecimento, e ele responde à pergunta que o HD44780 não sabia responder: **chegou?**

CONCEITO

E a mesma ideia resolve o problema maior. A escrita interna da memória leva cerca de 5 ms, durante os quais ela não responde a nada. Em vez de esperar 5 ms às cegas, o mestre simplesmente **pergunta**: emite um start e o byte de controle, e observa se veio reconhecimento. Se não veio, ainda está ocupada; tenta de novo.

É a mesma espera do encontro 3 com uma diferença de qualidade: lá o processador esperava o pior caso tabelado, aqui ele descobre o caso real. Numa escrita que termina em 3 ms, sobram 2 ms de trabalho útil que o HD44780 teria jogado fora.

A 100 kHz, cada byte ocupa o barramento por 90 μs. A transação inteira da figura — quatro bytes mais as condições — custa cerca de 400 μs, ou 1600 ciclos de máquina.

NOTA

Contra os 5200 do display e os 60 do conversor. E, como na serial, o processador só precisa aparecer entre os bytes.

4 Página e vida útil

Escrita por página. A memória tem um pequeno registrador interno — 8 ou 16 bytes, conforme a peça — e um único ciclo de gravação de 5 ms serve para tudo que estiver nele. Gravar oito bytes em uma página custa os mesmos 5 ms que gravar um; gravar oito bytes um a um custa 40 ms.

CONCEITO

O detalhe que morde: o contador de endereço dentro da página **dá a volta**. Escrever além do limite não avança para a página seguinte — sobrescreve o começo da mesma página.

A memória se gasta. Uma memória desse tipo suporta da ordem de um milhão de ciclos de escrita por posição; a interna do PIC, algo como cem mil.

ATENÇÃO

Se o programa gravar o alvo uma vez por segundo:

$$10^6 / 86\,400 \approx 11,6 \text{ dias}$$

Onze dias, e a posição está morta.

É a mesma conta da vida útil do relé no encontro 9, num componente sem partes móveis. E a solução tem a mesma forma: **escrever só quando o valor muda**, que era exatamente a estratégia adotada para o display no encontro 3.

Três problemas diferentes, uma resposta só.

5 O protocolo acima do protocolo

Tudo nesta aula foi dito sobre uma memória, e nada dela é sobre memória.

Existem **dois** protocolos empilhados. O I²C diz como os bytes andam pelos fios — início, endereço, reconhecimento, parada. O dispositivo diz o que os bytes **significam**, e isso é assunto da folha de dados dele, não da especificação do barramento.

CONCEITO

E a convenção de significado é quase universal entre dispositivos I²C: um **mapa de registradores**, alcançado por um byte de ponteiro. Escreve-se o número do registrador, emite-se start repetido, e lê-se.

Que é exatamente a figura da seção 3.3. O “endereço interno” da memória e o “número do registrador” de um sensor são o mesmo mecanismo com dois nomes.

Um sensor de temperatura I²C comum, o LM75A, tem quatro registradores:

Ponteiro	Registrador	Para que serve
0	temperatura	Dois bytes, mais significativo primeiro
1	configuração	Modo de operação, polaridade da saída de alerta
2	Thyst	Limite de rearme
3	Tos	Limite de disparo

Os dois últimos merecem atenção: Tos e Thyst são um limite superior e um limite de rearme, e o sensor tem um pino que comuta entre eles.

NOTA

Isso é o encontro 9 inteiro — controle liga-desliga com histerese — implementado em silício, dentro de um sensor de vinte reais. Ligar esse pino ao aquecedor e comparar com o termostato que a turma escreveu é a demonstração mais direta que existe da fronteira entre o que é firmware e o que é hardware.

Alongamento de relógio. Alguns dispositivos, quando ainda não têm a resposta pronta, seguram a linha SCL em nível baixo depois do reconhecimento. O mestre precisa perceber que o relógio não subiu e esperar.

ATENÇÃO

O módulo de hardware do PIC lida com isso. Um I²C escrito em software, que apenas gera bordas em intervalos fixos, não lida — ele segue em frente e lê lixo.

Nem todo dispositivo alonga; memórias em geral não, vários sensores sim. **Verifique na folha de dados do dispositivo, não na do barramento**, porque isso é comportamento do dispositivo.

Analogico contra digital, na mesma medição. Vale a comparação porque o resultado contraria o que a turma espera.

OPCIONAL

	LM35 + ADC de 10 bits	LM75A	MCP9808
Resolução	0,488 °C	0,125 °C	0,0625 °C
Exatidão	±0,5 °C do sensor, mais os erros da cadeia	±2 °C	±0,25 °C
Fiação	um canal analógico dedicado	dois fios compartilhados	dois fios compartilhados
Fio longo	o ruído vira erro de medida	irrelevante	irrelevante

O sensor digital barato é **pior em exatidão** que a cadeia analógica. O que ele compra é fiação e imunidade a ruído, não precisão — e a diferença entre o LM75A e o MCP9808 mostra que isso é característica da peça, não da tecnologia.

Vale também como fecho do encontro 4: se o problema fosse resolução, existe o ADS1115, que é um conversor de dezesseis bits acessado por este mesmo barramento. A pergunta “como consigo mais resolução” tinha, o tempo todo, uma resposta que não passava por melhorar o conversor interno.

Esta caixa pode ser pulada sem prejuízo para o restante do curso.

6 A colisão de pinos

O módulo de comunicação síncrona usa RB0 para SDA e RB1 para SCL. Esses dois pinos já apareceram duas vezes neste curso:

NESTE KIT

são as entradas de interrupção externa INT0 e INT1, candidatas naturais aos botões do encontro 7;

são canais analógicos, e portanto o PBAEN do encontro 4 volta a importar — quatro encontros depois, e pelo mesmo motivo.

A verificar antes do R11: como o teclado matricial e a memória convivem em PORTB, e se há chaves na placa que isolem a memória do barramento.

7 Previsão para o R11

PREVISÃO

P1. Escreva o alvo na memória, desligue a placa, ligue de novo. O que espera ver no display no primeiro segundo?

P2. Meça no osciloscópio o tempo de subida de SDA. Ele é igual ao de descida? Justifique antes de medir.

P3. Grave oito bytes um a um e depois os mesmos oito por página. Que razão entre os dois tempos você espera?

P4. Faça o programa gravar o alvo a cada volta do laço principal. Estime em quanto tempo a posição se esgotaria — e depois **não** deixe rodando.

P5. Compare o tempo de uma escrita na memória interna com o de uma escrita na externa. Qual é mais rápida, e por quê?

Leve esta folha preenchida. Este roteiro grava memória de verdade: use posições diferentes a cada tentativa e anote quais já foram usadas.

SEM NOTA

8 Exercícios

Exercício 11.1. Explique por que a sequência 0x55, 0xAA precisa rodar com as interrupções desabilitadas, e o que aconteceria sem isso.

TAREFA

Exercício 11.2. Um programa grava dezesseis bytes de configuração começando no endereço 0x00 de uma memória com página de oito bytes, usando uma única operação de escrita por página.

TAREFA

(a) O que fica gravado?

(b) Como corrigir?

(c) Quanto tempo leva a versão correta?

Exercício 11.3. Compare as duas memórias não voláteis quanto a: tempo de escrita, número de fios, quantidade de bits de endereço, e o que acontece se o dispositivo não responder.

TAREFA

Exercício 11.4. O termostato guarda o alvo e um contador de horas de funcionamento. Proponha uma estratégia de gravação que respeite a vida útil da memória, e justifique cada escolha.

TAREFA

Exercício 11.5. A folha de dados de um sensor informa que o endereço dele é 0x48. Um colega escreve `i2c_enviar(0x48)` logo após a condição de início e não recebe reconhecimento.

TAREFA

(a) Que dispositivo ele endereçou, e em que sentido?

(b) O que deveria ter enviado, para escrita e para leitura?

(c) O sintoma observado é distinguível de um resistor de elevação ausente?

No encontro 12: avaliação integradora II, cobrindo os encontros 7 a 11.

NOTA

E no encontro 13: controle embarcado na prática. As peças estão todas na mesa — uma planta que responde, um sensor que mede, uma chave que aplica potência, um registro que atravessa o desligamento, e um enlace que leva os dados para fora.

Falta usar os números que o R9 mediu para responder à pergunta que ele levantou: por que a temperatura oscila, e o que se faz a respeito.